

CAMPUS LOST & FOUND MANAGEMENT SYSTEM

C++ Object-Oriented Programming Lab Project Report

Submitted By:

Muhammad Hassan Raza (250901025)
Minahil Aamir (250901026)
Aliha Anwar Rana (250901030)
Amna Khurshid (250901041)

Course:

Object-Oriented Programming (OOP)

Instructor:

Miss Fizza

Department:

AI-02 (Computer Science)

Submission Date:

June 5, 2026

Table of Contents

- 1. Abstract**
- 2. Real-World Problems We Addressed**
- 3. System Functionality: Breaking Down Menu Options**
 - 3.1 Option 1: Report a Lost Possession
 - 3.2 Option 2: Log a Found Item Entry
 - 3.3 Option 3: View All Registered Logs
 - 3.4 Option 4: Run Match Identification Cross-Reference
 - 3.5 Option 5: Exit Application
- 4. OOP Concepts Integrated Into the Code**
 - 4.1 Abstraction
 - 4.2 Inheritance
 - 4.3 Polymorphism
 - 4.4 Encapsulation
 - 4.5 Dynamic Memory and Static Tracking
- 5. Expected Output**
- 6. Conclusion & Future Enhancements**

1. Abstract

On any busy college campus, students constantly lose track of personal belongings like keys, wallets, laptops, and student IDs. Relying on physical notice boards or chaotic group chats rarely works out well. To fix this common headache, we built a console-based Campus Lost & Found Management System in C++. By taking advantage of core Object-Oriented Programming (OOP) design patterns, we created a structured framework that handles logging these reports and automatically matches lost items with found ones to get them back to their rightful owners quickly.

2. Real-World Problems We Addressed

We designed this system to directly solve the typical issues found in manual or unorganized campus lost-and-found setups:

- **Scattered Information:** Instead of dealing with unorganized physical lost boxes or scattered notices, we built a centralized in-memory database array (`Item* itemDatabase[100]`) to store every single record in one predictable location.
- **No Way to Match Items:** Manual searching through logs is slow and inefficient. Our system implements an automatic tracking function (`crossReferenceDatabase()`) that instantly identifies matches by item name to link both parties together.
- **Communication Gaps:** Standard setups collect items but rarely note down how to contact the owner or the finder. Our application requires contact information for every single entry, letting the finder and owner connect directly without hurdles.

3. System Functionality: Breaking Down Menu Options

Our `main()` function drives the entire program using a clean do-while loop and an interactive menu. Here is exactly what happens when we press each button on the menu:

3.1 Option 1: Report a Lost Possession

- **What it does:** This option prompts the user to type in the item's name, its last known location, a physical description, their personal contact info, and any cash reward they want to offer.
- **Code Implementation:** It collects these inputs via `getline()` to handle spaces safely without breaking the input stream. It then dynamically instantiates a new `LostItem` object on the heap using the new keyword and passes it directly to the `registerItem()` function. The `itemType` is automatically assigned as 'Lost' behind the scenes by the constructor.

3.2 Option 2: Log a Found Item Entry

- **What it does:** This is used when someone finds an object on campus and wants to turn it in. It asks for the item name, where it was discovered, its current condition, the finder's contact details, and the specific locker or storage bin where it is being securely held.

- **Code Implementation:** Similar to Option 1, this captures the data and instantiates a FoundItem object on the heap. It assigns the itemType as 'Found' and stores the unique storage bin string before pushing the pointer into our system management array.

3.3 Option 3: View All Registered Logs

- **What it does:** This prints a clean, organized list of every single record currently stored in our system so staff or students can easily browse the layout.
- **Code Implementation:** The system first checks if totalItemsRegistered == 0 to prevent printing blank data headers. If data exists, it loops through the itemDatabase[] array and executes itemDatabase[i]->displayDetails();. Because displayDetails() is a virtual function, C++ figures out the exact object type at runtime, printing either a custom 'LOST ITEM RECORD' header with reward details or a 'FOUND ITEM RECORD' header with the holding bin location.

3.4 Option 4: Run Match Identification Cross-Reference

- **What it does:** This acts as our automated matching engine. It automatically cross-references the entire database to find matches, pulling up the contact details of both the owner and finder so they can quickly arrange a pickup.
- **Code Implementation:** The crossReferenceDatabase() function checks if there are fewer than 2 items to avoid useless processing loop runs. It then runs a nested for loop comparing every item against each other. It uses two strict filtering rules that we laid out in our code comments: (1) It completely skips matching items of the same type (ensuring itemType must be different, preventing a Lost item from matching another Lost item), and (2) It evaluates a direct text-matching rule on the names (getItemName() == getItemName()). When a match triggers true, it prints both sets of contact details side-by-side on the screen.

3.5 Option 5: Exit Application

- **What it does:** This safely shuts down the system, displays a final summary count of all items handled during the session, and exits the loop execution gracefully.
- **Code Implementation:** It prints out our shared global static counter, LostAndFoundSystem::totalItemsRegistered. Once the main menu loop terminates, the program triggers the LostAndFoundSystem class destructor. This destructor runs a loop across our pointer array and explicitly executes delete on every dynamic object, wiping our heap allocation clean and perfectly preventing any memory leaks before the program closes down.

4. OOP Concepts Integrated Into the Code

The architecture of this project relies entirely on the relationship between our classes to keep the system clean, modular, and maintainable. Here is how we applied the core OOP concepts:

4.1 Abstraction

We made Item an abstract base class by giving it a pure virtual function: `virtual void displayDetails() const = 0;`. This serves as our layout blueprint. It prevents anyone from creating a generic, incomplete 'Item' object without specifying if it is actually lost or found, hiding the structural implementation away from the main loop.

4.2 Inheritance

Our LostItem and FoundItem classes inherit publicly from the Item base class. This allows them to seamlessly reuse the protected core data variables (itemName, location, description, contactInfo) without us having to duplicate or rewrite those lines of code in both classes. We then extended them with unique features like rewardAmount for lost entries and storageBin for found ones.

4.3 Polymorphism

By defining our database array using base pointers (`Item* itemDatabase[100]`), we can mix both lost and found records inside a single array. Dynamic binding ensures that whenever we call `displayDetails()`, the program executes the correct overridden version of the child class at runtime based on the actual object sitting in that memory slot.

4.4 Encapsulation

We restricted direct external access to our critical variables by declaring them private or protected. Whenever the cross-reference engine needs to check item details for matches, it is forced to use our public read-only getter functions like `getItemName()` and `getItemType()`, keeping our core database variables completely secure from accidental or unwanted alterations.

4.5 Dynamic Memory and Static Tracking

We used heap allocation via the `new` keyword inside the main menu inputs to create fresh item instances dynamically. To manage this safely, our destructor loops through the array and calls `delete` to free up the memory when the system shuts down. Additionally, we used a static `int` `totalItemsRegistered` counter that remains shared across the entire class engine, acting as a reliable global ledger counter.

5. Expected Output

```
spgym.f4n' '--stdout=Microsoft-MIEngine-Out-wtu52ilp.s4a' '--stderr=Mic
.exe' '--interpreter=mi'

=====

      CAMPUS LOST AND FOUND MANAGEMENT SYSTEM
=====

----- MAIN MENU -----
1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application
Select Operation Option (1-5): 1

--- ENTER LOST ITEM METADATA ---
Identify Item Name (e.g. Ring, Phone, Wallet): Phone
Estimated Location Lost At: bench outside mess
Describe Appearance Details: black Iphone
Provide Personal Contact Information: 123@gmail.com
Enter Offered Monetary Reward (or None): 1000

>> System Alert: Lost Item Entry Logged Successfully!

----- MAIN MENU -----
1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application
Select Operation Option (1-5): 1

--- ENTER LOST ITEM METADATA ---
Identify Item Name (e.g. Ring, Phone, Wallet): Umbrella
Estimated Location Lost At: class room 1234
Describe Appearance Details: Dark Blue
Provide Personal Contact Information: Aliha@gmail.com
Enter Offered Monetary Reward (or None): 500,000
```

```

>> System Alert: Lost Item Entry Logged Successfully!

----- MAIN MENU -----
1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application
Select Operation Option (1-5): 2

--- ENTER FOUND ITEM METADATA ---
Identify Item Name Found: Umbrella
Location Identified At: Class room ground floor
Describe Object Physical Condition: blue
Provide Finder's Contact Information: finder@gmail.com
Assigned Secure Holding Storage Bin #: none

>> System Alert: Found Item Entry Logged Successfully!

----- MAIN MENU -----
1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application
Select Operation Option (1-5): 3

=====
CURRENT DATABASE REGISTERED RECORDS
=====

--- [LOST ITEM RECORD] ---
Item Name      : Phone
Lost From      : bench utside mess
Appearance     : black Iphone
Contact Info: 123@gmail.com
Offered Reward: 1000
-----

```

--- [LOST ITEM RECORD] ---

Item Name : Umbrella
Lost From : class room 1234
Appearance : Dark Blue
Contact Info: Aliha@gmail.com
Offered Reward: 500,000

--- [FOUND ITEM RECORD] ---

Item Name : Umbrella
Found Near : Class room ground floor
Description : blue
Claim Contact: finder@gmail.com
Holding Bin : none

----- MAIN MENU -----

1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application

Select Operation Option (1-5): 4

=====
RUNNING CROSS-REFERENCE SEARCH ENGINE
=====

🎯 MATCH DETECTED FOR VALUE: 'Umbrella'

👤 Lost By : Aliha@gmail.com

👤 Found By : finder@gmail.com

📍 Locations : class room 1234 vs Class room ground floor

```
📌 Locations : class room 1234 vs Class room ground floor
-----

----- MAIN MENU -----
1. Report a Lost Possession
2. Log a Found Item Entry
3. View All Registered Logs
4. Run Match Identification Cross-Reference
5. Exit Application
Select Operation Option (1-5): 5

Thank you for using Campus Lost and Found System. Terminating Program...
Total Database Logs Processed: 3
PS C:\STUDY STUFF\C++ Codes>
```

6. Conclusion & Future Enhancements

Our console system effectively applies academic programming concepts to manage a real campus problem. The code utilizes runtime polymorphism to handle diverse records smoothly, features robust input filtering via `cin.fail()` to ensure stability against bad inputs, and implements a built-in cleanup destructor to handle dynamic memory safely.

While it currently operates entirely within local RAM (meaning data clears out every time the program restarts), the underlying code is organized so that features like file persistence (using `fstream` to save reports to a text file) or a graphical user interface (GUI) can be added down the road without rewriting any of our core object models.